

CG作业8报告

李惟乐 PB23050929

内容

- [算法原理](#)
- [代码实现与结果](#)

算法原理

1. 半隐式时间积分的弹簧系统

半隐式积分有以下迭代：

$$\mathbf{v}^{(n+1)} = \mathbf{v}^{(n)} + h\mathbf{M}^{-1}(\mathbf{f}_{\text{int}}(\mathbf{x}^{(n)}) + \mathbf{f}_{\text{ext}}) \quad (1)$$

$$\mathbf{x}^{(n+1)} = \mathbf{x}^{(n)} + h\mathbf{v}^{(n+1)} \quad (2)$$

其中的 $\mathbf{M} \in \mathbb{R}^{3n \times 3n}$ 为系统的质量矩阵，为对角矩阵（每个顶点对应的 $\mathbb{R}^{3 \times 3} = \text{mass_per_vertex}$ 乘3x3单位矩阵）。

$\mathbf{f}_{\text{int}}$ 为弹性力，若设 E 为系统的弹性势能，则有：

$$\mathbf{f}_{\text{int}} = -\nabla E$$

单个弹簧 i 的弹性势能为：

$$E_i = \frac{k}{2}(\|\mathbf{x}_{i1} - \mathbf{x}_{i2}\| - L)^2$$

记 $\mathbf{x}_i := \mathbf{x}_{i1} - \mathbf{x}_{i2}$

总能量为：

$$E = \sum_i E_i = \sum_i \frac{k}{2}(\|\mathbf{x}_i\| - L)^2$$

进一步求得梯度：

$$\nabla E = \sum_i k(\|\mathbf{x}_i\| - L) \frac{\mathbf{x}_i}{\|\mathbf{x}_i\|} \quad (3)$$

为加强结果收敛性，为速度乘上 `vel *= damping` 以模拟阻尼。

2. 隐式时间积分的弹簧系统

隐式积分有以下迭代：

$$\begin{aligned} \mathbf{x}^{n+1} &= \mathbf{x}^n + h\mathbf{v}^{n+1} \\ \mathbf{v}^{n+1} &= \mathbf{v}^n + h\mathbf{M}^{-1}(\mathbf{f}_{\text{int}}(\mathbf{x}^{n+1}) + \mathbf{f}_{\text{ext}}) \end{aligned} \quad (4)$$

将下式带入上式，得到：

$$\mathbf{x}^{n+1} = \mathbf{x}^n + h\mathbf{v}^n + h^2\mathbf{M}^{-1}(-\nabla E(\mathbf{x}^{n+1}) + \mathbf{f}_{\text{ext}}) \quad (5)$$

定义 $\mathbf{y} := \mathbf{x}^n + h\mathbf{v}^n + h^2\mathbf{M}^{-1}\mathbf{f}_{\text{ext}}$

可以得到：

$$\frac{1}{h^2}\mathbf{M}(\mathbf{x}^{n+1} - \mathbf{y}) + \nabla E(\mathbf{x}^{n+1}) = \mathbf{0} \quad (6)$$

令 $g(\mathbf{x}) = \frac{1}{2h^2}(\mathbf{x} - \mathbf{y})^\top \mathbf{M}(\mathbf{x} - \mathbf{y}) + E(\mathbf{x})$

对其求梯度，得到：

$$\nabla g(\mathbf{x}) = \frac{1}{h^2}\mathbf{M}(\mathbf{x} - \mathbf{y}) + \nabla E(\mathbf{x}) \quad (7)$$

注意此处对角阵 $\mathbf{M} \in \mathbf{R}^{n \times n}$

因此求解 (6) 式，等价于优化该问题：

$$\nabla g(\mathbf{x}) = \frac{1}{h^2}\mathbf{M}(\mathbf{x} - \mathbf{y}) + \nabla E(\mathbf{x})$$

采用牛顿法求解，有以下迭代：

$$\mathbf{x}^{n+1} = \mathbf{x}^n - (\nabla^2 g(\mathbf{x}^{(n)}))^{-1} \nabla g(\mathbf{x}) = \mathbf{x}^n - (\nabla^2 g)^{-1} \nabla \mathbf{g}$$

接下来求能量 g 的Hessian矩阵 $\nabla^2 g$

对单根弹簧：

$$\mathbf{H}_i = \nabla^2 E_i = k \frac{\mathbf{x}_i \mathbf{x}_i^\top}{\|\mathbf{x}_i\|^2} + k \left(1 - \frac{L}{\|\mathbf{x}_i\|} \right) \left(\mathbf{I} - \frac{\mathbf{x}_i \mathbf{x}_i^\top}{\|\mathbf{x}_i\|^2} \right)$$

总体的Hessian阵 $\mathbf{H} \in \mathbf{R}^{3n \times 3n}$ 由各 $\mathbf{H}_i$ 组装。将 $\mathbf{H}$ 分块为 n 行 n 列。对于任意弹簧，若其连接的结点为 第 i 个和第 j 个，Hessian记为 $\mathbf{H}_e$

则分块坐标 (i, i) 和 (j, j) 的分块矩阵加 $\mathbf{H}_e$

分块坐标为 (i, j) 和 (j, i) 的分块矩阵减 $\mathbf{H}_e$

最后组装出 g 的Hessian：

$$\nabla^2 g = \frac{1}{h^2}\mathbf{M} + \mathbf{H}$$

带回得到：

$$\begin{aligned} \mathbf{x}^{n+1} &= \mathbf{x}^n + \Delta \mathbf{x} \\ &= \mathbf{x}^n - (\nabla^2 g)^{-1} \nabla g(\mathbf{x}^n) \end{aligned}$$

需解方程为：

$$\nabla^2 g \Delta \mathbf{x} = -\nabla g$$

将右侧展平即可求解方程组。

为保证 $\nabla^2 g$ 正定，采用以下方法：

在计算弹簧能量的Hessian时，当 $L_i > \|\mathbf{x}_i\|$ 时，令第 i 根弹簧 $\mathbf{H}_i$ 近似为 $\mathbf{H}_i \approx k \frac{\mathbf{x}_i \mathbf{x}_i^\top}{\|\mathbf{x}_i\|^2}$ 。

另外令固定点所在的 $\nabla^2 g$ 分块为单位阵，同时令右侧的 $-\nabla g$ 的对应项为零。

求解后，更新速度：

$$\mathbf{v} = (\mathbf{x}^{\text{new}} - \mathbf{x}^{\text{old}})/h$$

3. 与球的交互

增加一个碰撞力：

$$\mathbf{f} = k^{\text{penalty}} \max(sr - \|\mathbf{x} - \mathbf{c}\|, 0) \frac{(\mathbf{x} - \mathbf{c})}{\|\mathbf{x} - \mathbf{c}\|}$$

其中 $\mathbf{c}$ 为球心， r 为球半径， s 为半径的一个放大系数（如让 $s=1.1$ ），用于在实际未发生接触时就产生接触力以减少视觉上的穿透， k^{penalty} 为可调参数。

代码实现与结果

对 `MassSpring` 类进行了如下修改：

- 新增元素 `std::vector<int> dirichlet_vertices`，便于快速查找固定点
- 新增函数 `bool isFixedPoint(int i) const` 用于查询序号 `i` 的顶点是否为固定点

另外新增函数：

- 在构建Hessian阵 $\mathbf{H}$ 时，为方便分块构建，建立以下函数：

```
// push 3x3 matrix to triplet list, row and col are the block index
void pushMatrix3dtoTripletList(std::vector<Eigen::Triplet<double>>& tripletList, int row,
int col, const Eigen::Matrix3d& block)
{
    for (int i = 0; i < 3; ++i)
    {
        for (int j = 0; j < 3; ++j)
        {
            tripletList.emplace_back(3 * row + i, 3 * col + j, block(i, j));
        }
    }
}
```

- 计算单个弹簧的Hessian阵

```
Eigen::Matrix3d MassSpring::computeHessianByEdge(double stiffness, const Edge& e, int i)
{
    Eigen::Vector3d x_i_1 = x.row(e.first);
    Eigen::Vector3d x_i_2 = x.row(e.second);
    Eigen::Vector3d x_i = x_i_1 - x_i_2;
    double norm_x_i = x_i.norm();

    Eigen::Matrix3d H_i = Eigen::Matrix3d::Zero();

    if (norm_x_i < E_rest_length[i])
    {
```

```

        H_i = stiffness * (x_i * x_i.transpose()) / (norm_x_i * norm_x_i);
    }
    else
    {
        H_i = stiffness * (x_i * x_i.transpose()) / (norm_x_i * norm_x_i) +
            stiffness * (1 - E_rest_length[i] / norm_x_i) *
            (Eigen::Matrix3d::Identity() - x_i * x_i.transpose() / (norm_x_i * norm_x_i));
    }
    return H_i;
}

```

gif 均存在同一文件夹中

比如 0_600_0_003

第一个 0 表示是隐式积分（1 为半隐式积分）

第二个 600 表示 stiffness

第三个和第四个表示 h 为 0.003

1. 半隐式时间积分的弹簧系统

进行了以下参数的测试：

(stiffness, h)

(600, 0.001)

(600, 0.002)

(600, 0.003)

(550, 0.003)

可以看出：

- 随着步长 `h` 的拉大，动画速度加快，这是固定采样率，不同采样步幅所决定的。
- 当 `h == 0.03`，布料运动发生了爆炸，半隐式时间积分对步长较为敏感，收敛性一般
- 当保持 `h == 0.03`，减小 `stiffness`，布料运动恢复正常，说明 `stiffness` 和 `h` 共同影响着收敛性。

2. 隐式时间积分的弹簧系统

进行了以下参数的测试：

(stiffness, h)

(100, 0.002)

(100, 0.01)

(100, 0.02)

(100, 0.04)

(200, 0.01)

(600, 0.01)

(600, 0.03)

除了与之前相同的结论，还可以看出：

- `stiffness` 越大，布料达到相对稳态的速度相对更快
- 隐式积分收敛性明显好于半隐式积分：
 - 当 `stiffness == 600`, `h == 0.03` 时才出现爆炸现象，提升了约十倍。
 - 如果令 `stiffness == 100`，则在 `h == 0.04` 时才出现爆炸

3. 与球的交互

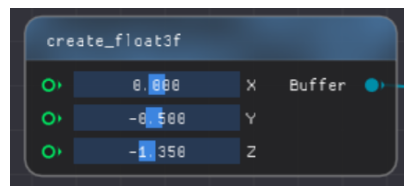


图1: 球体计算位置

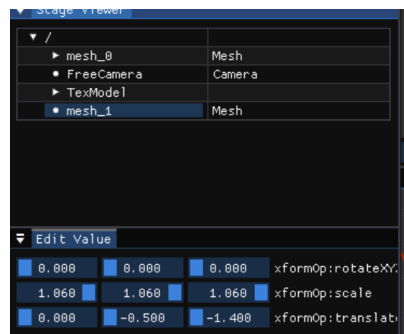


图2: 球体mesh位置

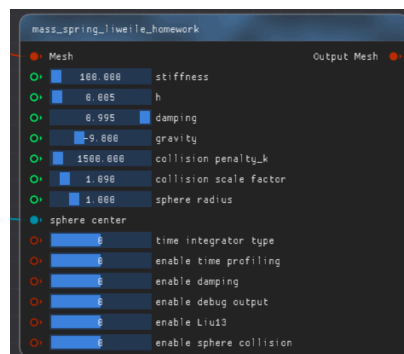


图3: Collision1参数设置

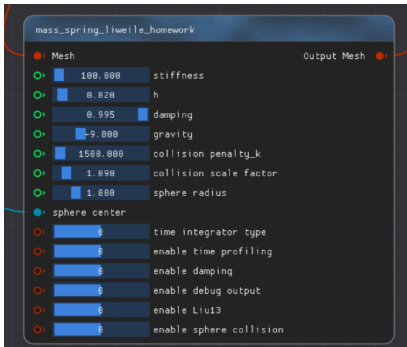


图4: Collision2参数设置

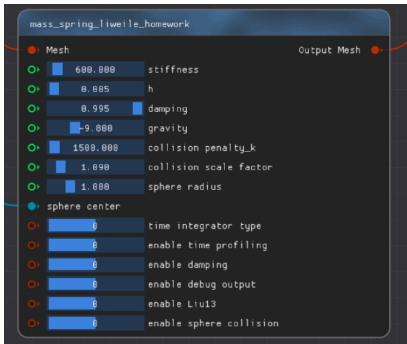


图5: Collision3参数设置

当 `stiffness` 较大的时，布料明显更厚重，更快趋于稳态，但 `h` 需要调低，否则容易发生爆炸。另外当图形面数增加时，`stiffness` 和 `h` 也需要留出更大的余量。